SLIIT UNI
THE KNOWLEDGE UNIVERSITY

Faculty of Computing

Year 2 Semester 1 (2025)

IT2011 - Artificial Intelligence and Machine Learning — Lab Sheet 03

Exploratory Data Analysis and Machine Learning Pipeline Practical

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is the process of examining datasets to understand their key characteristics using visual and statistical methods. It involves summarizing the data, identifying patterns, and revealing relationships between variables. Common visualization tools include histograms, box plots, and scatter plots. EDA is a crucial step before applying machine learning, as it helps define the problem clearly and guides model selection. It provides valuable insights and supports informed decision-making.

Task 01

Given a car dataset with over 10,000 rows and more than 10 features including Engine Fuel Type, Engine HP, Transmission Type, Highway MPG, and City MPG, perform exploratory data analysis (EDA) to make it suitable for machine learning modeling.

Open the Jupyter Notebook and create a new Notebook. Name the Notebook as car-data_eda. You need to perform the following operations to complete the task.

1. **Import the Python Libraries**

   (a) Pandas
   Pandas is a fast, powerful, and open-source Python library built on top of NumPy, designed for data manipulation and real-world data analysis. It provides efficient handling of missing data, flexible reshaping and pivoting of datasets, and supports size mutability. These features make Pandas an excellent tool for managing and transforming data effectively.

(b) Numpy
NumPy (short for Numerical Python) is a free and open-source Python library commonly used in science, engineering, and data analysis. It provides powerful tools for working with numbers, especially through its ndarray, a special type of array that can hold data in multiple dimensions (like rows and columns). NumPy also includes many built-in functions that can quickly and efficiently perform calculations on these arrays.

(c) Seaborn
Seaborn is a popular Python library for data visualization, built on top of Matplotlib. It provides a simple and user-friendly way to create attractive and meaningful statistical graphs. Seaborn works well with Pandas dataframes, which makes it easy to explore and visualize data quickly and efficiently.

(d) Matplotlib
Matplotlib is a powerful and flexible open-source library in Python used for creating a wide range of data visualizations. It allows to turn raw data into clear and informative plots, such as line charts, bar graphs, scatter plots, and more. It helps make data easier to analyze and understand through visual representation.

```
import pandas as pd
import numpy as np
import seaborn as sns                    #visualisation
import matplotlib.pyplot as plt          #visualisation
%matplotlib inline
sns.set(color_codes=True)
```

%matplotlib inline will display the plot below the code, i.e. inline with the code.

2. **Load the dataset**

```
df = pd.read_csv("cars_data.csv")
```

pd.read_csv() is a function from the pandas library. It is used to read a CSV file and load the data into a DataFrame.

car_data.csv': This is the name of the CSV file you want to read. It should be in the same folder as your Python script or notebook unless you provide the full path.

df is a variable that stores the data that was read. It becomes a DataFrame, which is like a table (with rows and columns) in Python.

3. **3. Display the first and the last 5 rows of the data**

```
#displaying first five rows

display(df.head())

# or df.head()
```

```
#displaying first five rows

display(df.tail())

# or df.tail()
```

4. **Check for the Types of the Data in the Dataset**

   We check the data types of each column to ensure they are appropriate for analysis and visualization. For example, the MSRP or car price might sometimes be stored as a string (text) instead of a numerical value. In such cases, we need to convert it to an integer or float to perform calculations or create plots. However, in this dataset, the price values are already in the correct integer format, so no conversion is necessary.

```
df.dtypes
```

5. **Remove the irrelevant columns**

   Datasets often contain columns that are irrelevant to the current analysis or modeling task. In such cases, dropping these unused columns is the best way to simplify the dataset. For the task given, columns like Engine Fuel Type, Market Category, Vehicle Style, Popularity, Number of Doors, and Vehicle Size are not relevant, so they will be removed.

```
df = df.drop(['Engine Fuel Type', 'Market Category', 'Vehicle Style', 'Popularity', 'Number of Doors', 'Vehicle Size'], axis=1)
df.head(5)
```

6. **Renaming the columns**

   Column renaming is useful when the column names are difficult to understand by the user.

```
df = df.rename(columns={"Engine HP": "HP", "Engine Cylinders": "Total Cylinders",
                        "Transmission Type": "Mechanism",
                        "Driven_Wheels": "Drive Mode","highway MPG": "MPG-H",
                        "city mpg": "MPG-C", "MSRP": "Price" })

df.head(5)
```

7. **Check for Duplicate Rows**

   i. Check the dimension of the DataFrame

```
df.shape
```

   ii. Find the duplicate rows

```
df.count()          # Used to count the number of rows
```

```
duplicate_rows_df = df[df.duplicated()]
print("number of duplicate rows: ", duplicate_rows_df.shape)
```

   df.duplicated() - This returns a Boolean Series (True/False) indicating whether each row in the DataFrame is a duplicate of a previous row. df[df.duplicated()] - This filters the DataFrame and returns only the rows that are duplicates (excluding the first instance). duplicate_rows_df.shape - This returns the shape of the new DataFrame (number of duplicate rows, number of columns).

   iii. **Remove the duplicate rows**

```
df = df.drop_duplicates()
df.head(5)
```

8. Drop the Missing or Null Values

   There are various techniques to fill the missing or null values, since the given dataset has only a few missing values, we drop them. However, it is recommended to drop the missing values.

```
print(df.isnull().sum())
```

   The above command determines the number of missing values in the dataset. Then use the command below to drop the columns.

```
df = df.dropna()    # Dropping the missing values.
df.count()
```

9. Outliers Detection

   An outlier is a data point that significantly differs from other observations in the dataset. These values can be either unusually high or low compared to the rest of the data. Detecting and removing outliers is an important step in data preprocessing, as they can negatively impact the accuracy and performance of machine learning models.

   In this analysis, Interquartile Range (IQR) method is used to identify and remove outliers. The IQR technique is a reliable and commonly used approach that identifies values lying far outside the middle range of the data.

Outliers can also be visually identified using box plots. Shown below are box plots for variables such as MSRP, Cylinders, Horsepower, and Engine Size. In each plot, the points that appear outside the whiskers of the box represent potential outliers. By detecting and addressing these outliers, we aim to improve the quality and reliability of the dataset before proceeding to the modeling stage.

```
sns.boxplot(x=df['Price'])
plt.show()
```

```
sns.boxplot(x=df['HP'])
plt.show()
```

```
sns.boxplot(x=df['Total Cylinders'])
plt.show()
```

Next calculate the first (Q1) and third (Q3) quartiles and find the Inter Quartile Range (IQR). This operation can be performed only for the columns having numeric data.

```
# Step 1: Identify numeric columns
numeric_cols = df.select_dtypes(include='number').columns

# Step 2: Compute Q1, Q3, and IQR for only numeric columns
Q1 = df[numeric_cols].quantile(0.25)
Q3 = df[numeric_cols].quantile(0.75)
IQR = Q3 - Q1

# Step 3: Filter rows that are NOT outliers in any numeric column
# Keep only rows where all numeric values are within the IQR bounds
condition = ~((df[numeric_cols] < (Q1 - 1.5 * IQR)) | (df[numeric_cols] > (Q3 + 1.5 * IQR))).any(axis=1)

# Step 4: Apply this condition to the full DataFrame (all columns)
df_cleaned = df[condition]

# Step 5: Check the new shape
print(df_cleaned.shape)
```

10. Visualization Plots

    i. Scatter Plot

```
fig, ax = plt.subplots(figsize=(10,6))
ax.scatter(df['HP'], df['Price'])
ax.set_xlabel('HP')
ax.set_ylabel('Price')
plt.show()
```

    ii. Histogram

```
df.Make.value_counts().nlargest(40).plot(kind='bar', figsize=(10,5))
plt.title("Number of cars Vs Make")
plt.ylabel('Number of cars')
plt.xlabel('Make');
plt.show()
```

iii. Heat Maps

```
# First, filter only numeric columns for correlation
numeric_df = df.select_dtypes(include=['float64', 'int64'])

plt.figure(figsize=(10,5))
c = numeric_df.corr()  # Calculate correlation only on numeric columns
sns.heatmap(c, cmap="BrBG", annot=True)

# Uncomment to display the plot
plt.show()
```

Machine Learning Pipeline

Data preprocessing is a key step in data mining that involves converting raw data into a format that is easy to interpret and analyze. In real-world scenarios, data is often incomplete, inconsistent, or missing important patterns and may contain various errors. Preprocessing addresses these challenges effectively by cleaning and organizing the data, making it suitable for further analysis.

Task 02

Using the knowledge obtained from the previous task, perform the data preprocessing operation for the **Question 01: Uncovering Learning Pattern** and develop a simple machine learning model

1. Open the Jupyter Notebook and create a new Notebook. Name the Notebook as data_preprocessing.
2. Import the Python Library (Hint: Pandas)
3. Load the Data set in Pandas DataFrame
4. Display the first and the last 5 rows of the data
5. Display the column names of the DataFrame

```
# Program to print all the column name of the dataframe

print(list(df.columns))
```

6. Display the summary of the DataFrame

```
df.info()
```

The info() function provides a concise summary of a DataFrame, displaying key details such as the data type of each column, the RangeIndex (total number of rows), the list of columns, the count of non-null entries in each column, and the overall memory usage of the DataFrame.

7. Display the Descriptive Statistical Features of the DataFrame

```
df.describe()
```

The .describe() function gives a quick summary of the numerical columns in the DataFrame. It shows the basic statistics like average, minimum, maximum, and standard deviation for the numbers in the table

8. Handling Missing Data

   (a) Finding the missing values

   To determine the number of missing values in the dataset *(Hint: isnull().sum())*

   (b) Fill missing Age and Quiz Score with their mean values

```
df.fillna(df['Age'].mean(), inplace=True)
```

```
df.fillna(df['Quiz Score'].mean(), inplace=True)
```

9. Encode Categorical Data

```
# Gender and Result is encoded using Label Encoding

df["Gender"] = df["Gender"].map({"Male": 1, "Female": 0})

df["Result"] = df["Result"].map({"Pass": 1, "Fail": 0})
```

10. Add a New Feature "EngagementLevel"

```
df["Engagement Level"] = df["Lessons Completed"] / (df["Time Per Week"] + 0.0001)
```

11. Normalize numerical features using MinMaxScaler

```
# import MinMaxScaler Library to complete this task
from sklearn.preprocessing import MinMaxScaler
```

```
## Normalization

scaler = MinMaxScaler()

scale_cols = ["Age", "Lessons Completed", "Quiz Score", "Time Per Week", "Engagement Level"]

df[scale_cols] = scaler.fit_transform(df[scale_cols])

df.head()
```

First import the MinMaxScaler class from scikit-learn. The MinMaxScaler is used to scale (normalize) numeric data so that all the values are in the range between 0 and 1.

Next, create a 'scaler' object using the MinMaxScaler class. This object will be used to fit and transform the data. Then provide the list of the column names in the DataFrame (df) that you want to scale. fit_transform() first calculates the min and max values of each column. Then, it scales each value using the formula and replaces the original columns in df with the scaled values.

12. Preview the final preprocessed dataset *(Hint: Use print())*

A Simple Machine Learning Model

1. Prepare features (X) and label (y) for model training

```
X = df.drop(columns=["StudentID", "Result", "Country"])  # Keep Country unencoded

y = df["Result"] # Target Variable
```

df.drop(columns=...): This removes specific columns from the DataFrame.

We need to drop:

"StudentID" – because it's just an ID and not useful for prediction.
"Result" – because this is the target variable (label), so we don't want it in the input features.
"Country" – possibly because it's categorical

X = ... stores the remaining columns as the feature set (independent variables) – the input that will be used to train the model.

y becomes the target/output variable (also called the label), which you're trying to predict using the features in X.

2. Split data into training and testing sets

```
#import Libarary

from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

First import the train_test_split function from scikit-learn. It is used to split the data into training and testing sets.
X: Your features (input data).
y: Your labels (what you want to predict – Pass/Fail).

This data is split into:

- X_train: 80% of the input data, used to train the model.
- X_test: 20% of the input data, used to test how well the model performs.
- y_train: 80% of the target results, matched to X_train.
- y_test: 20% of the target results, matched to X_test.

test_size=0.2 → 20% for testing, 80% for training.
random_state=42 → Ensures the split is the same every time you run the code (for reproducibility).

3. Build and train a Random Forest Classifier

```
#import library

from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier(random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

To build the Random Forest Classifier, you need to imports the RandomForest-Classifier from scikit-learn.

model.fit() trains the model using your training data.

Now that the model is trained, this model.predict() predictions on the test data (X_test). The results are saved in y_pred, which contains predicted outcomes (Pass/Fail) for the test set.

4. Model Evaluation

```
# import library
from sklearn.metrics import classification_report, accuracy_score


print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

First import the following tools to measure the accuracy:

- accuracy_score: Measures how often the model was correct.
- classification_report: Gives detailed results like precision, recall, and F1-score for each class (Pass and Fail).

print("Accuracy:", accuracy_score(y_test, y_pred)) compares the actual results (y_test) with the model's predictions (y_pred) and displays the overall percentage of correct predictions.

print(classification_report(y_test, y_pred, target_names=["Fail", "Pass"])) shows detailed metrics for each class:

- Precision: Of all predicted Passes, how many were correct?
- Recall: Of all actual Passes, how many were found?
- F1-score: A balance between precision and recall.

5. Check each Feature Importance

This displays a chart that shows which features (like Age, Quiz Score, etc.) were the most useful for the model in predicting Pass or Fail

```
#import libraries
import matplotlib.pyplot as plt
import seaborn as sns

importances = model.feature_importances_
features = X.columns

plt.figure(figsize=(8, 5))
sns.barplot(x=importances, y=features)
plt.title("Feature Importances")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```

matplotlib.pyplot and seaborn are Python libraries used to create charts and graphs. seaborn builds on top of matplotlib and makes the plots look nicer.

- model.feature_importances returns the importance score of each feature in the trained Random Forest model.
- X.columns gives the names of the features, so that we can label the chart correctly.
- plt.figure(figsize=(8, 5)) sets the size of the plot to be 8 inches wide and 5 inches tall.
- sns.barplot(x=importances, y=features) creates a horizontal bar chart showing how important each feature is.
    - x = importances → bar lengths show the importance.
    - y = features → labels each bar with the feature name.
- plt.tight_layout() adjusts spacing so nothing overlaps.
- plt.show() displays the final plot.